other basis, we first apply a gate that rotates that basis onto the computational basis and then measure. This is exactly what happened at the end of the previous article. The circuit that prepares the Bell states—a Hadamard gate on qubit 0 followed by a CNOT with qubit 0 as control and qubit 1 as target—carries the four computational basis states onto the four Bell states. Run backwards, CNOT first and then the Hadamard gate, it carries each Bell state back to a distinct computational basis state that the hardware can read: $|\Phi+\rangle$ to 00, $|\Phi-\rangle$ to 01, $|\Psi+\rangle$ to 10 and $|\Psi-\rangle$ to 11, with Qiskit printing qubit 1 on the left. A Bell measurement is therefore nothing more mysterious than a change of basis followed by an ordinary measurement. For a single qubit, the gate that rotates the Hadamard basis onto the computational basis is the Hadamard gate itself, which is why the basis bears its name: H turns $|+\rangle$ into $|0\rangle$ and $|-\rangle$ into $|1\rangle$.

Measuring in a different basis

The program *qiskit13.py*, shown below, prepares the states $|+\rangle$ and $|-\rangle$, and measures each of them twice: once in the computational basis (the Z basis) and once in the Hadamard basis (the X basis).

```
1.  from qiskit import QuantumCircuit, transpile
2.  from qiskit_aer import AerSimulator
3.  sim = AerSimulator( )
4.  for state in ["+", "-"]:
5.      for basis in ["Z", "X"]:
6.          qc = QuantumCircuit(1, 1)
7.          qc.h(0)
8.          if state == "-":
9.              qc.z(0)
10.         if basis == "X":
11.             qc.h(0)
12.         qc.measure(0, 0)
13.         compiled = transpile(qc, sim)
14.         counts = sim.run(compiled, shots=1000)
            .result( ).get_counts( )
15.         print(f"|{state}> measured in {basis}
            basis:", counts)
```

Now let us understand how the program works. Lines 4 and 5 loop over the states and bases. Line 7 prepares $|+\rangle$ with a Hadamard gate, and Lines 8 and 9 turn it into $|-\rangle$ with a Pauli-Z gate when required—recall from the third article that Z leaves $|0\rangle$ alone and multiplies $|1\rangle$ by −1. Lines 10 and 11 contain the only new idea: when we want to measure in the X basis, we apply one more Hadamard gate before the measurement. Lines 12 to 15 measure, run 1000 shots and print the counts. Activate *qiskit_env* and run the program with the command *python qiskit13.py*. The output is shown in Figure 2. Recall that the counts obtained on your terminal will differ slightly from run to run.

```
(qiskit_env) # python qiskit13.py
|+> measured in Z basis: {'0': 499, '1': 501}
|+> measured in X basis: {'0': 1000}
|-> measured in Z basis: {'0': 514, '1': 486}
|-> measured in X basis: {'1': 1000}
```

Figure 2: Output of the program *qiskit13.py*

Measured in the Z basis, $|+\rangle$ and $|-\rangle$ behave identically, each producing roughly 500 zeros and 500 ones. This is the same disappointment we met with $|\Phi+\rangle$ and $|\Phi-\rangle$ in the previous article. Measured in the X basis, however, $|+\rangle$ yields 0 in all 1000 shots, and $|-\rangle$ yields 1 in all 1000 shots. Orthogonal states can always be told apart by a suitable measurement, but not by every measurement. Randomness is therefore not a property of a quantum state alone; it is a property of a state together with the question we ask it. Try preparing $|0\rangle$ and measuring it in the X basis, and you will find that the roles are reversed: the state that is certain in the Z basis behaves like a fair coin in the X basis. Take it as a challenge: to make *qiskit13.py* prepare $|0\rangle$ and $|1\rangle$ instead of $|+\rangle$ and $|-\rangle$, make cosmetic changes to the state labels in Lines 4 and 8, replace the Z gate in Line 9 with the gate that turns $|0\rangle$ into $|1\rangle$ (Hint: read the second article in this series), and then remove Line 7.

The Hadamard operator, up close

In the first article, we met the Hadamard gate as a 2×2 matrix: $1/\sqrt{2}$ times the matrix with rows (1, 1) and (1, −1), shown again in Figure 3. Look at its columns. The first column is $|+\rangle$ and the second is $|-\rangle$, so H sends $|0\rangle$ to $|+\rangle$ and $|1\rangle$ to $|-\rangle$. Moreover, multiplying H by itself gives the identity matrix, $H^2 = I$, so the Hadamard gate is its own inverse and sends $|+\rangle$ back to $|0\rangle$ and $|-\rangle$ back to $|1\rangle$. The Hadamard gate is therefore far more than a quantum coin-flipper: it is a translator between two languages, converting the computational basis into the Hadamard basis and back again. This is exactly why it appears in Lines 10 and 11 of *qiskit13.py*. Notice, too, the lonely minus sign in the bottom-right corner. Without it, the two columns would be identical rather than orthogonal, and the matrix would not be a valid quantum gate; that minus sign is what allows amplitudes to cancel.

Figure 3 also shows a few more identities. The Hadamard matrix can be written as $H = (X + Z)/\sqrt{2}$, a perfectly balanced blend of two Pauli gates. More importantly, HXH = Z and HZH = X. In words, a bit flip viewed through Hadamard glasses becomes a phase flip, and a phase flip becomes a bit flip. This explains the experiment of the third article, in which the sequence Hadamard, Pauli-Z, Hadamard took the qubit from $|0\rangle$ to $|1\rangle$ with certainty: the three gates together are simply an X gate in disguise. On the Bloch sphere, the Hadamard gate is a 180° rotation about the diagonal axis halfway between the x-axis and the z-axis. That rotation